

Industrial Safety Intelligence

A Compound-Risk Detection Platform for Heavy Industry

Technical Documentation

Sai Venkat Ashok Reddy Aditya Reddy Revanth

ETAI Hackathon
Abstract

Indian heavy industry records over 6,500 fatal workplace accidents a year. The recurring pattern in post-incident investigations is not absent instrumentation but absent integration: gas readings, work permits, shift logs and CCTV each sit in a separate system, and no layer fuses them before an incident. This document describes the implementation of a platform that detects *compound risk* — a hazardous process condition co-occurring with authorised human activity, a combination no single-sensor threshold flags — and turns each detection into an auditable, regulation-grounded response. It covers the heterogeneous graph neural network that scores plant zones, the multi-agent pipeline that correlates those scores against permits and drafts reports, the retrieval layer that grounds every regulatory claim in a real corpus, the computer-vision detectors, the API and real-time streaming design, the operator interface, and the measured results against a single-sensor baseline. All inference, including the report-writing language model, runs on-premise.

Contents

1	Introduction	2
1.1	The failure mode this system targets	2
1.2	Why a threshold model is structurally unable to do this	2
1.3	Design constraints	2
1.4	Scope and honest positioning	3
2	System Architecture	3
2.1	Layered decomposition	3
2.2	Request lifecycle	3
2.3	Multi-tenancy	4
2.4	Technology selection	4
3	Data Layer	4
3.1	The benchmark corpus	4
3.2	Relational schema	6
3.3	Seeding	6
4	The Compound-Risk Graph Network	6
4.1	Why the graph is heterogeneous	7
4.2	Handling time	7
4.3	Message passing	8
4.4	Training	8
4.5	The baseline we compare against	8
4.6	Explaining a score	8
5	The Agent Pipeline	9
5.1	Why LangGraph and not a prompt chain	9
5.2	Node one: scoring	9
5.3	Node two: permit correlation	10
5.4	Node three: orchestration	10

6	Regulatory Grounding	10
6.1	Corpus and chunking	11
6.2	Embedding and storage	11
6.3	A deployment problem worth documenting	11
6.4	How grounding is enforced	11
7	Computer Vision	12
7.1	Four detectors	12
7.2	Sessions and streaming	12
7.3	Separating detection from interpretation	13
7.4	An honest note on the demonstration frames	13
8	Backend and Real-Time Layer	13
8.1	Result caching	13
8.2	Device and IoT integration	13
8.3	Streaming the live plant	14
8.4	Alerting and evidence	14
9	Operator Interface	14
9.1	The plant map	15
9.2	Views	15
9.3	Theming and interface state	15
9.4	Chat memory	15
10	Evaluation	16
10.1	Detection quality	16
10.2	False negatives at matched false-alarm budgets	16
10.3	Zone localisation	16
10.4	Lead time, stated honestly	16
10.5	On the saturated metrics	17
11	Deployment and Operations	17
11.1	One-command run	17
11.2	GPU and platform notes	17
11.3	Kubernetes and CI	17
11.4	Two operational lessons	18
12	Limitations and Further Work	18
12.1	What the evaluation does not establish	18
12.2	Known gaps	18
12.3	Security posture	18
12.4	Closing	19

1 Introduction

1.1 The failure mode this system targets

On 20 January 2025, eight workers died at the Visakhapatnam Steel Plant when entrapped gases triggered an explosion in a coke oven battery. The facility was not unequipped: it operated gas detectors, a permit-to-work regime, and a SCADA installation. Investigation reporting established that pressure-sensor warnings existed in the data before the event, but that no layer connected those readings to the operational decision to place people in the area.

This pattern repeats. A 2024 FICCI survey found that over 60% of large Indian industrial facilities coordinate between their own digital safety tools through manual handoffs. The DGFASLI figure of 6,500+ fatal workplace accidents in FY2023 excludes most mining and construction activity, so the true total is higher.

The technical observation underlying this project is that these incidents are rarely caused by a single variable crossing a single threshold. They are caused by *combinations*:

- a gas concentration that is elevated but individually sub-alarm, *and*
- an active confined-space permit placing workers in that same zone, *and*
- a maintenance deferral that left a relief path degraded.

Each fact, examined alone, is unremarkable. A permit is normal. A mildly raised reading is normal. A threshold monitor watching any one channel stays silent. We refer to the conjunction as **compound risk**, and detecting it is the system’s core function.

1.2 Why a threshold model is structurally unable to do this

A single-sensor alarm evaluates each channel independently against a fixed bound. To catch a compound condition in which no individual channel is extreme, its thresholds must be lowered until they fire on ordinary variation. Our measured baseline (Section 10) demonstrates exactly this trade. To reach full recall it operates at 0.37 precision, which is another way of saying it alarms almost constantly. That is not a tuning failure but a representational one. The information distinguishing a hazardous plant state from a benign one lives in the *relationships* between channels, zones, permits and people; a per-channel model has no mechanism to represent a relationship.

This motivates a graph formulation, in which zones, sensors, permits and workers are typed nodes and the process topology supplies the edges (Section 4).

1.3 Design constraints

Four constraints shaped the implementation, and each is visible in the architecture.

On-premise inference. Plant telemetry and CCTV cannot be sent to a third-party API, and a safety layer must not fail when the site’s internet link does. Every model in the pipeline executes locally: the graph network, the embedding model, the vision detectors and the report-writing language model. No component makes an outbound call at inference time.

Auditability over accuracy alone. A safety decision that cannot be explained after the fact is inadmissible in an investigation. Every stage therefore emits an audit record: which sensors drove a score, which permit was correlated, which regulatory passage was retrieved, and who acknowledged the resulting alert.

No invented regulation. A language model asked to cite safety law will fabricate plausible section numbers. The system removes the opportunity: the model is only ever given text retrieved from a real corpus and instructed to quote from it (Section 6).

Deployability without an integration project. A platform requiring months of systems integration is not adoptable. Facility onboarding is template-driven, and device integration is a single authenticated JSON POST (Section 8).

1.4 Scope and honest positioning

The system is a working prototype evaluated on a synthetic benchmark derived from the Tennessee Eastman process simulation, with real regulatory documents and real fine-tuned vision models. Two claims are deliberately *not* made.

First, we do not claim earlier detection. On our benchmark the median confirmation latency is identical to the baseline’s. What the graph model provides is precision and correct zone attribution, not lead time; Section 10 presents this directly rather than selecting a more flattering metric.

Second, the CCTV evidence uses images from a public hard-hat dataset. We have no real plant footage. The inference path is identical for a live camera, and connecting one is what the device layer exists to do, but the demonstration frames themselves are staged and are labelled as such in the interface.

2 System Architecture

2.1 Layered decomposition

The platform is organised into five layers. The separation is not cosmetic: each layer has a distinct failure mode and a distinct rate of change, and the boundaries are where the persistence and audit points sit.

1. **Ingestion.** Process telemetry, work permits, shift records, IoT sensor pushes and camera streams. Heterogeneous in schema and in arrival rate.
2. **Perception.** Two independent model families reduce raw input to structured findings: the graph network scores zones from telemetry, and the vision detectors emit events from frames.
3. **Reasoning.** A three-stage agent pipeline correlates those findings against operational context (who holds a permit, who is on site) and decides an escalation level.
4. **Grounding.** Retrieval over a regulatory corpus supplies the citations that make a generated report defensible.
5. **Response.** Alerting with an acknowledgement trail, evidence export, and the operator interface.

2.2 Request lifecycle

Tracing one scenario execution end to end makes the interaction concrete. A request to POST `/run/{run_id}` proceeds as follows.

1. **Cache check.** The server looks for a previously persisted result for this run. Because a full pass costs roughly a minute of local LLM inference, a repeat request returns the stored result immediately; `?force=true` bypasses this.
2. **State assembly.** Sensor windows, permits and worker-presence records for the run are loaded and packed into a typed `PipelineState` dictionary.
3. **Graph scoring.** The first agent builds a heterogeneous graph from that state, runs the network, and writes a compound-risk score and a baseline score for all seven zones, plus per-sensor saliency.

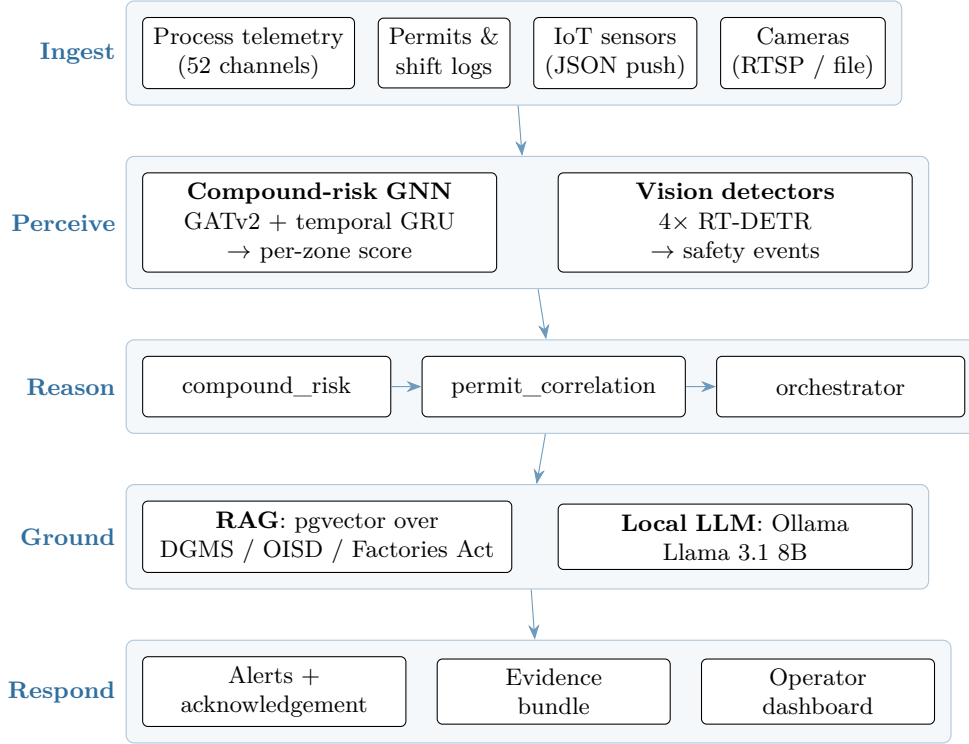


Figure 1: Layered architecture. Perception reduces raw signals to structured findings; reasoning fuses them with operational context; grounding constrains what the language model may assert.

4. **Permit correlation.** The second agent joins high-scoring zones against active permits and emits violations carrying a severity band.
5. **Orchestration.** The third agent retrieves regulatory passages relevant to the violation, prompts the local LLM to draft an incident report constrained to those passages, and sets an escalation level.
6. **Persistence and alerting.** Scores, violations, citations, report text and the audit log are committed in one transaction. If the escalation is **alert** or **emergency**, an alert row is created and an optional webhook fires asynchronously.

2.3 Multi-tenancy

Every table representing real-world state carries a `facility_id`. This is not speculative generality: one deployment is expected to monitor several plants, each with its own zone layout, permit stream and devices. A consequence is that zone identity is per-facility rather than global, and the interface has to respect that. The plant map renders from the facility’s own stored zone coordinates, falling back to a fixed schematic layout only for the built-in demonstration plant.

2.4 Technology selection

3 Data Layer

3.1 The benchmark corpus

Real plant telemetry from an incident is, for obvious reasons, not something a research project can obtain. We therefore built the benchmark on the Tennessee Eastman process simulation, a chemical plant model that has served as a fault-detection reference since the early 1990s. It gives us 52 measured channels: 41 process measurements, labelled `XMEAS(1)` through `XMEAS(41)`, and 11 manipulated variables, `XMV(1)` to `XMV(11)`.

Component	Implementation	Rationale
Graph model	PyTorch 2.12, PyTorch Geometric 2.8	Heterogeneous graphs and attention are first-class; attention weights double as explanations.
Agent pipeline	LangGraph 1.2, SQLite check-pointing	Explicit state machine with a persisted trace, rather than an opaque prompt chain.
Language model	Ollama, Llama 3.1 8B	Runs on commodity on-site hardware; swappable without code change.
Retrieval	sentence-transformers MiniLM-L6-v2 (384-d) with pgvector	Embeddings live beside relational data; no separate vector store to operate.
Vision	RT-DETR (Ultralytics 8.4), OpenCV	Transformer detector; anchor-free and robust in crowded frames.
API	FastAPI 0.138, Pydantic v2	Typed request models; OpenAPI schema generated rather than maintained by hand.
Database	PostgreSQL 16 + pgvector, SQLAlchemy 2, Alembic	One store for both relational and vector workloads.
Frontend	React 19, Vite 8	Plant map drawn as SVG, so geometry is data rather than an image.
Streaming	Server-Sent Events; MJPEG	Unidirectional and proxy-tolerant; no WebSocket infrastructure required.

Table 1: Principal technology choices and the reasoning behind each.

What the simulation does not give us is the human layer. A Tennessee Eastman fault is purely a process fault; nobody holds a permit and nobody is standing next to the reactor. Since the whole premise of this project is the interaction between process state and human activity, we generate that layer ourselves and pair it with each run.

Every run is one episode of 120 samples at a three-minute interval, so six hours of plant time. Runs are catalogued in `manifest.csv` with the fields shown below, and the telemetry itself is stored per run as Parquet.

Field	Meaning
<code>run_id</code>	12-character identifier, also the Parquet filename
<code>scenario_id</code>	Which of the five hazard scenarios this run belongs to
<code>condition</code>	compound or normal ; the supervision signal
<code>zone</code>	The zone a fault was injected into, for compound runs
<code>permit_type</code>	Permit class active during the run
<code>compound_active</code>	Whether the process fault and the human activity actually overlap
<code>ground_truth_onset_sample</code>	Sample index at which the injected fault begins
<code>path</code>	Location of the run’s Parquet file

Table 2: Run manifest schema. Ground-truth onset is what makes lead-time measurement possible.

The five scenarios were chosen because each mirrors a documented accident pattern: reactor heat-removal loss, condenser pressure build-up, deferred maintenance on a relief path, feed system disturbance, and a common-cause utility failure. Each scenario contributes both compound runs, where a fault coincides with an active permit, and normal control runs where the same permit exists but the process stays healthy. That pairing matters. Without control runs sharing the permit, a model could reach high accuracy by learning nothing more than “a permit is open”.

Permits and shift records live in their own Parquet files keyed by `run_id`, each carrying validity windows. The windows are load-bearing at evaluation time: a permit that becomes active at sample 60 must not be visible to the model when it is scoring sample 40. Section 10 returns

to this point, because an early version of the replay code leaked that information and produced flattering lead-time numbers that were not real.

3.2 Relational schema

Everything relational moved into PostgreSQL. Bulk numeric data did not; sensor windows stay on disk as Parquet and the `runs` table simply points at them. Splitting on that line keeps the database useful for querying without turning it into a poorly optimised array store.

Seventeen tables are in use. They group into five families:

- **Tenancy and topology.** `facilities` anchors everything. `zones` holds each monitored area with its map coordinates, unique per facility.
- **Benchmark data.** `runs`, `permits`, `worker_presence`.
- **Pipeline output.** `zone_risk_scores`, `permit_violations`, `incident_reports`, `audit_log`. These are what make incident history real rather than something discarded after each request.
- **Devices and telemetry.** `devices`, `sensor_readings`, `vision_sessions`, `vision_events`, `alerts`.
- **Retrieval.** `rag_documents` and `rag_chunks`, the latter carrying a `vector(384)` column indexed by pgvector.

Two schema decisions are worth explaining because they were not obvious at the time.

Embeddings live in the main database. An earlier version kept the retrieval index as a NumPy array loaded into each process. Moving it into a `vector` column means the corpus can be joined against other tables, and it removes a second piece of infrastructure from the deployment. The dimension is fixed at 384 to match the embedding model, and the two have to be changed together.

Alerts carry their own acknowledgement fields. `acknowledged_by` and `acknowledged_at` sit directly on the alert rather than in a separate event log. The question an investigator asks is “did somebody see this, and when”, and answering it should not require reconstructing state from an event stream.

3.3 Seeding

`db.seed` loads the flat-file benchmark into the schema and is safe to re-run. It truncates the tables it owns with `CASCADE`, which is necessary because pipeline output rows accumulate real foreign keys back to `runs` and `zones` during normal use, and an ordered per-table delete would trip over them.

Two things are deliberately exempt from the wipe. Facilities are preserved, and so are API keys, because a key handed to somebody has to keep working across a reseed. The seed also loads the regulatory corpus and computes chunk embeddings, which is the slowest part of a first start.

4 The Compound-Risk Graph Network

This is the component that does the actual detection, and the one where the modelling choice matters most. The task is to assign each of the seven zones a risk score in $[0, 1]$ from a window of telemetry plus whatever human activity is recorded against that zone.

4.1 Why the graph is heterogeneous

A sensor reading, a work permit and a worker are different kinds of thing. They have different feature widths, different semantics, and different relationships to a zone. Forcing them into one node type would mean padding them into a shared feature space and hoping the network sorts it out.

We use a heterogeneous graph instead, with six node types.

Node type	Input dim	What it represents
sensor	5	One process channel, summarised over the window
sensor_cluster	sequence	A group of channels belonging to one zone
zone	7	A monitored area; one-hot zone identity
permit	5	An active work permit, one-hot over four permit types
presence	1	Whether workers are recorded in the zone
worker	2	Staffing detail for the zone
plant	1	A single root node the zones report to

Table 3: Node types and their input feature widths.

Channels are grouped into six clusters by the equipment they instrument. The reactor cluster, for instance, is XMEAS(7), XMEAS(8), XMEAS(9), XMEAS(21) and XMV(10). Each cluster attaches to the zone it monitors. Keeping individual `sensor` nodes alongside the clusters is what lets the explanation layer rank single channels rather than reporting a whole cluster as responsible.

Edges are typed and, with one exception, bidirectional so that information can travel in both directions during message passing. Sensors feed clusters, clusters monitor zones, permits authorise zones, presence and workers occupy them, and every zone reports to the plant root.

The exception is the process topology. Zone-to-zone edges follow the physical flow of material:

`feed` $\rightarrow$ `reactor` $\rightarrow$ `condenser` $\rightarrow$ `separator` $\rightarrow$ `stripper`, with `separator` $\rightarrow$ `compressor` $\rightarrow$
`reactor` closing the recycle loop.

These six edges are the reason risk can propagate between zones at all. A disturbance in the reactor is a different proposition depending on what sits downstream of it, and the topology is how the model gets access to that fact.

4.2 Handling time

An early version of the model mean-pooled each channel’s window into summary statistics before message passing. That discards the shape of a trajectory, which is exactly the information that separates a process drifting into trouble from one sitting at a steady offset.

The current design keeps the sequence. Each cluster’s 30-sample window, covering 90 minutes, is fed to a GRU:

```
self.temporal_encoder = nn.GRU(MAX_CLUSTER_SENSORS, hidden, batch_first=True)
...
_, h_n = self.temporal_encoder(data["sensor_cluster"].x)
x_dict["sensor_cluster"] = F.relu(h_n.squeeze(0))
```

The final hidden state becomes the cluster’s node embedding entering message passing. Note that `sensor_cluster` is the one node type with no input projection layer, since the GRU already produces a vector of the right width.

4.3 Message passing

Every other node type is projected to a 32-dimensional hidden space by its own linear layer. Two rounds of heterogeneous message passing follow, each a `HeteroConv` wrapping one `GATv2Conv` per edge type, with two attention heads and sum aggregation.

Two hops is a deliberate choice rather than a default. One hop would let sensor evidence reach its own zone and no further. Two lets a zone see its process-flow neighbours after those neighbours have themselves absorbed their sensor evidence, which is what “risk propagating through the plant” means concretely.

Residual connections wrap both rounds:

```
h1 = self.conv1(x_dict, edge_index_dict)
x_dict = {k: F.relu(h1[k] + x_dict[k]) for k in h1}
h2 = self.conv2(x_dict, edge_index_dict)
x_dict = {k: h2[k] + x_dict[k] for k in h2}
```

These are not decoration. Without them the second convolution recomputes each zone purely from leaf embeddings that only depended on the previous zone state, and the model collapses to predicting the constant class prior. We observed that collapse directly during development, and the residuals were added in response to it.

A small MLP head maps each zone embedding to a logit, giving seven predictions per graph.

4.4 Training

Training runs for 60 epochs with Adam at a learning rate of 10^{-3} , weight decay 10^{-4} , batch size 32, and a fixed seed of 0.

Two details had a disproportionate effect on whether the model trained at all.

Normalisation. Raw channel values span roughly 0 to 4500. Fed in unnormalised, `GATv2` training stalls at the entropy of the class prior. Features are z-scored per node type, with statistics computed on the training split only and then applied to validation and test. Computing them over the full dataset would leak.

Class weighting. Compound-positive zones are a minority of all zone labels, and an unweighted model finds it profitable to predict zero everywhere. The positive class is weighted by 6.0 in the loss.

The split is 70/15/15, stratified, and performed at the level of runs rather than timesteps. Splitting on timesteps would place samples from the same episode on both sides of the boundary, which inflates results without meaning anything.

4.5 The baseline we compare against

The comparison model is deliberately not a strawman. It is the zone-aware version of what plants actually run: for each zone, take that zone’s own sensors, z-score them against statistics fitted on normal runs, and alarm when any channel exceeds four standard deviations. It is evaluated on the identical test split, at each run’s true hazard zone. Section 10 reports what it achieves.

4.6 Explaining a score

Two mechanisms make a score inspectable, and both matter for a safety audit.

Per-sensor saliency. A vanilla-gradient method backpropagates a single zone’s logit to the input features and ranks channels by gradient magnitude. One backward pass per explanation, which is cheap enough to serve on hover in the interface. It replaced an earlier display that listed a zone’s sensors without actually computing anything from the model, a substitution we considered dishonest once noticed.

Flow attention. `flow_attention()` reads the GATv2 attention weights on the zone-to-zone edges at the second convolution, averaged over heads. This answers “which upstream neighbour is this zone listening to”.

One caveat is worth stating plainly, because it would otherwise mislead. GAT attention normalises over each target’s incoming edges. A zone with exactly one upstream neighbour therefore always shows an attention of 1.0, regardless of whether anything interesting is happening. The interface compensates by scaling the displayed weight by the source zone’s risk score, so a pipe only lights up when a genuinely risky zone is the one being attended to.

5 The Agent Pipeline

A risk score on its own is not actionable. Somebody still has to ask whether anyone is in that zone, whether the work happening there is authorised, how serious the combination is, and what the regulations say about it. Those questions are handled by three agents running in sequence.

5.1 Why LangGraph and not a prompt chain

The pipeline is a `StateGraph` with three nodes and a linear edge structure:

```
graph.add_node("compound_risk", compound_risk_node)
graph.add_node("permit_correlation", permit_correlation_node)
graph.add_node("orchestrator", orchestrator_node)
graph.add_edge("compound_risk", "permit_correlation")
graph.add_edge("permit_correlation", "orchestrator")
graph.add_edge("orchestrator", END)
```

A linear sequence could obviously have been three function calls. What the graph buys us is the checkpointer. Every node transition is written to a SQLite database at `agents/checkpoints/audit_trail.sql`, which means the complete state at every stage of every run is replayable after the fact. For a safety system that is not a convenience, it is the audit trail. An investigator can ask what the model saw at the moment it made a call, and get an answer that does not depend on anyone having logged the right thing.

All three nodes read and write one shared `PipelineState`, a `TypedDict` declared in `agents/state.py`. Fixing that contract early let the model, the correlation logic and the frontend be built against a stable shape instead of guessing at each other’s output. Nodes append to the audit log rather than overwriting it, so the trail accumulates as the state moves through.

5.2 Node one: scoring

The first node loads the trained network and produces, for all seven zones, a compound-risk score, the baseline score for comparison, and the contributing sensors from the saliency pass.

One implementation detail is worth recording because it cost real debugging time. The checkpoints were trained on Apple Silicon, so they carry MPS storage tags. Loading them on a CPU-only or CUDA machine fails outright unless the load is pinned:

```
torch.load(REPO_ROOT / "models" / "gnn" / "checkpoint.pt", map_location="cpu")
```

Both the weights and the normalisation statistics need it.

5.3 Node two: permit correlation

This node is rule-based, and that is a design decision rather than an unfinished piece of work. It is the step that converts a model probability into a statement a safety officer can act on: not “the reactor scores 0.93” but “a confined-space permit is active in a zone the model scores 0.93”. A judge or an auditor can verify that logic by reading it. Learning it would make the most legally consequential step in the pipeline the least inspectable one.

Severity comes from banded thresholds:

≥ 0.9 critical ≥ 0.7 high ≥ 0.5 medium otherwise low

Two refinements sit on top of the bands, both added after reviewing output that read wrongly.

PPE evidence is corroborating, not causal. Camera evidence of missing helmets is appended to a violation’s reason text, but it does not by itself push the severity to critical. A violation escalated only by PPE is capped at `high` unless the sensor-driven risk is already at or above 0.9 on its own. Without the cap, a missing hard hat and an extreme process excursion produced identically worded critical alerts, which is not a distinction a control room can afford to lose.

Unauthorised presence has a floor. A person detected in a zone with no permit covering it is a compliance issue regardless of what the process is doing, so severity scales with the zone’s current risk but never falls below `medium`.

5.4 Node three: orchestration

The final node decides escalation, retrieves regulatory grounding, and drafts the report.

Escalation is a straight band over the maximum zone score, with the levels `none`, `monitor` at 0.5, `alert` at 0.7 and `emergency` at 0.9. If nothing crosses the floor, the node exits immediately with `none` and never invokes the language model, which keeps a quiet plant cheap to evaluate.

When it does escalate, the node retrieves the top three regulatory passages for the situation (Section 6) and builds a prompt containing the zone scores, the violations and those passages. The model is instructed to quote only from what it was given.

Generation goes to a local Ollama endpoint running Llama 3.1 8B. One parameter is worth explaining. On an 8 GB consumer GPU the full model does not fit alongside the vision and graph models, so `num_gpu` caps how many layers are offloaded and the remainder runs on CPU:

```
payload = {"model": model or OLLAMA_MODEL, "prompt": prompt, "stream": False}
if effective_num_gpu:
    payload["options"] = {"num_gpu": effective_num_gpu}
```

The default of 20 layers was tuned for that card. On a machine with more VRAM it can be raised, and on Apple Silicon running natively it is set high enough to place the whole model on the GPU. The same helper is reused by the operator chat, which may point at an entirely different endpoint and model, so the offload setting is applied only when the default local endpoint is the target.

A full pass takes roughly a minute on CPU-bound hardware, almost all of it in generation. The API caches results per run for exactly this reason, and the interface exposes an explicit re-run rather than silently recomputing.

6 Regulatory Grounding

Ask a language model what the Factories Act requires before hot work and it will produce a confident answer with a section number attached. The answer will often be wrong and the

section number frequently invented. For a system whose output may end up in a regulatory filing, that behaviour is disqualifying.

The response here is structural rather than a matter of prompt wording. The model is never asked to recall regulation. It is handed retrieved text and told to quote from it, so a fabricated citation has nowhere to come from.

6.1 Corpus and chunking

The corpus is real regulatory material covering Indian industrial safety: DGMS technical circulars, OISD standards, and the Factories Act. `rag/ingest.py` parses these into chunks with the page number and a citation label preserved on each one, since a citation that cannot be located in the source document is not much better than an invented one. The seeded corpus produces 205 chunks across three source documents.

6.2 Embedding and storage

Chunks are embedded with `all-MiniLM-L6-v2` from sentence-transformers, giving 384 dimensions. The model is small, fast on CPU, and adequate for retrieving short regulatory passages. Vectors are stored in the `rag_chunks` table in a pgvector `vector(384)` column.

The dimension is coupled to the model in two places, the schema and the ingest code, and changing one without the other produces an error at insert time rather than silently degraded retrieval.

Retrieval is cosine similarity, expressed through pgvector's distance operator, with similarity recovered as $1 - \text{distance}$:

```
query_vec = _model.encode(query, normalize_embeddings=True).tolist()
rows = (db.query(RagChunk, RagChunk.embedding.cosine_distance(query_vec).label("dist"))
        .order_by("dist").limit(k).all())
```

6.3 A deployment problem worth documenting

The embedding model was originally fetched from HuggingFace on first use. This works on a developer machine and fails badly in deployment. On a fresh container the first pipeline run would attempt the download mid-request, and where outbound TLS from the container is blocked or throttled the request did not fail cleanly. It hung, holding a file lock, while every subsequent request queued behind it. Diagnosing it required dumping the stack of the running server, which showed the process parked inside a HuggingFace file lock rather than doing anything model-related.

The fix was to vendor the model into the repository at `models/embedding/all-MiniLM-L6-v2`, alongside the graph checkpoints, and have the retriever prefer the local copy:

```
source = str(_LOCAL_MODEL_DIR) if _LOCAL_MODEL_DIR.exists() else "all-MiniLM-L6-v2"
_model = SentenceTransformer(source)
```

The container entrypoint also pre-warms the model before the API starts accepting traffic, so the cost is paid at startup instead of inside somebody's first request. The falling-back branch is kept for development machines that would rather pull from the hub.

This is a small change with a disproportionate effect: it is the difference between `docker compose up` working on a clean machine and appearing to hang forever.

6.4 How grounding is enforced

Retrieved passages enter the prompt as a labelled block, and the system instruction is explicit that citations may come only from that block. The same discipline applies in the operator chat,

where the retrieved sources are additionally surfaced in the interface under each answer, so a reader can expand the citation and see the passage the claim rests on.

The guarantee this provides is narrow and worth stating precisely. It does not mean the model’s interpretation of a regulation is correct. It means the regulation it quotes exists, is reproduced verbatim, and can be traced to a page in a real document. Interpretation remains a human responsibility, which is the correct division for a compliance workflow.

7 Computer Vision

Cameras contribute the part of the picture telemetry cannot see: whether people are present, whether they are wearing what they should be, and whether something has visibly gone wrong.

7.1 Four detectors

Each detector is an RT-DETR model served through Ultralytics, wrapped behind a common interface in `vision/detectors/base.py` so the session runner can treat them uniformly.

Detector	Weights	Emits
PPE	<code>rt detr_ppe.pt</code>	Head versus helmet detections, per person
Zone intrusion	<code>rt detr-1.pt</code> (stock)	Person count, cross-checked against permits
Fall / man-down	<code>rt detr_fall.pt</code>	Fall events
Fire / smoke	<code>rt detr_fire_smoke.pt</code>	Fire and smoke regions

Table 4: The four detectors. Only zone intrusion uses stock weights; the rest are fine-tuned.

RT-DETR was chosen over a YOLO-family detector mainly because it is anchor-free and holds up better in crowded frames, which is the normal condition on a work site. Three of the four sets of weights are fine-tuned on domain data; person detection uses the stock model, since detecting a person is exactly what it is already good at.

Models load lazily and behind a lock:

```
_model_lock = threading.Lock()
...
if _ppe_model is None:
    from ultralytics import RTDETR
    _ppe_model = RTDETR(str(PPE_WEIGHTS))
```

Loading four detectors eagerly would add substantial time to startup and consume VRAM that the language model also wants. Loading them on first use means a deployment that never opens a camera never pays for them.

7.2 Sessions and streaming

A vision session is created against a source, runs inference on a background thread, and keeps the most recent annotated frame plus an event list. Clients pull the video as MJPEG:

```
yield (b"--frame\r\nContent-Type: image/jpeg\r\n\r\n" + frame + b"\r\n")
```

MJPEG is not efficient, but it is displayable by a plain `<img>` tag with no client-side player, and it degrades sensibly when a frame is slow. Since the browser cannot attach custom headers to an image request, the API key travels as a query parameter on this endpoint, which is the standard accommodation for the case.

Demo clips and live cameras run the same code path. The only difference is the source handed to OpenCV and whether the video loops at the end.

7.3 Separating detection from interpretation

One decision in `bake_demo_frames.py` is worth drawing out because it reflects a general principle in the system.

The `bake` step runs the models and caches two things per frame: the PPE detector's output, and the zone-intrusion detector's raw person count. It deliberately does *not* cache whether an intrusion occurred. Whether a detected person constitutes unauthorised entry depends on which permits were active during that particular run, which is run-specific rather than image-specific. Caching the judgement would bake one run's permit context into an artefact reused across every run.

So the model output is cached and the cross-check happens live, using the cached count. It costs no extra inference and keeps the two concerns separate: the detector reports what it sees, the correlation layer decides what it means.

7.4 An honest note on the demonstration frames

We have no real plant CCTV. The demonstration frames come from a public hard-hat dataset, manually selected because the automatic filter, matching only on class labels, happily returned office interiors and photographs of handshakes that technically contained a head and no helmet.

Two frames are baked: one violation frame and one compliant frame. Every compound scenario therefore displays the same violation image. This is a limitation of the demonstration data rather than a defect in the pipeline, and the interface labels these frames as staged. The inference path for a live camera is identical, and connecting one is what the device layer exists to do.

8 Backend and Real-Time Layer

The API is intentionally thin. Domain logic lives in `agents/`, `models/` and `vision/`; the server adapts it to HTTP. It is organised as six routers mounted on one FastAPI application, covering devices, live risk streaming, benchmarks, alerts, chat and facility management, alongside the scenario endpoints in the main module.

Every endpoint accepts an optional `facility_id` and falls back to the demonstration plant when it is absent. That parameter is exposed even where the interface does not yet surface it, so the API is genuinely multi-tenant rather than merely having a multi-tenant schema.

8.1 Result caching

A pipeline run costs about a minute, almost entirely in language-model generation. Early on, every click re-ran everything, which made the interface feel broken.

Results are now persisted and replayed. The cache is keyed off the most recent `IncidentReport` for a run, and related rows are gathered using a timestamp cutoff a few seconds before it. That cutoff exists because rows from a single pass are committed in one transaction but receive per-row default timestamps milliseconds apart, with scores and audit entries written before the report. Anchoring slightly earlier captures the whole pass. It is safe because two runs of the same scenario are always separated by far more than the window.

The response carries a `cached` flag, the interface displays it, and an explicit re-run control forces recomputation. Users are told what they are looking at rather than being left to guess whether a fast response means stale data.

8.2 Device and IoT integration

The integration contract is deliberately minimal, because a platform requiring a vendor gateway does not get adopted. A device sends one JSON POST:

```
curl -X POST http://host:8000/iot/readings \
-H 'Content-Type: application/json' \
-H 'X-Device-Token: isi_dev_...' \
-d '{"device_id": "...", "value": 42.5}'
```

Anything that speaks HTTP qualifies: a PLC gateway, a microcontroller, a phone. Each sensor receives its own ingest token when registered, so a compromised device can be revoked without touching the others.

Cameras register with a source type of webcam index, RTSP URL, or file. The distinction that matters is what “online” means. Rather than reporting registration state, a connection test opens the source with OpenCV, reads an actual frame, and returns it as a base64 snapshot with the measured latency:

```
cap = cv2.VideoCapture(source)
ok, frame = cap.read() if cap.isOpened() else (False, None)
```

Sensor health follows the same principle and is derived from data freshness. A sensor that has not reported within two minutes reads as offline regardless of what its last manual test said.

A background thread generates readings for devices registered as simulated, writing through the same table as real hardware. This keeps the demonstration populated with moving data without a separate mock path that could drift from the real one.

8.3 Streaming the live plant

Continuous mode replays a run’s telemetry through the same scoring path used by time replay, one frame per tick, over Server-Sent Events. The trace is computed once and cached, so the stream itself is a timer over precomputed frames rather than repeated inference.

SSE was chosen over WebSockets because the data only travels one way, browsers reconnect automatically, and it passes through proxies without special configuration. The client is a plain `EventSource`. As with the video stream, the key rides as a query parameter, since `EventSource` cannot set headers either.

8.4 Alerting and evidence

Escalations at `alert` or `emergency` create an alert row and fire an optional webhook. The webhook runs on a daemon thread and swallows its own failures by design: notification delivery must never slow down or break risk detection.

Acknowledgement records who and when. That turns “somebody probably saw it” into a fact with a name and a timestamp attached, which is the question that actually gets asked after an incident.

The evidence bundle is a self-contained HTML page assembled server-side: zone scores against baseline, permit violations, the CCTV frame embedded as a data URI, the report with its citations, and the agent audit trail. Printing it to PDF produces the regulator handoff. We chose this over generating PDFs directly to avoid a reporting dependency for something the browser already does well.

9 Operator Interface

The interface is a React 19 application built with Vite. There is no component framework. For a dashboard whose central element is a custom schematic, a component library would mostly have been overhead.

9.1 The plant map

The map is hand-drawn SVG rather than an image with hotspots, which means zone geometry is data the application can reason about. Zones are positioned from stored coordinates, with a fixed schematic layout used for the demonstration plant so it reads as an organised process rather than a scatter of boxes: material flows left to right across the middle, the recycle compressor sits above, and the control room is docked aside since it is a monitoring area rather than part of the process line.

Risk is drawn as a continuous gradient from green through amber to red instead of four discrete bands, so a zone at 0.55 is visibly distinct from one at 0.68. Hard bands made different situations look identical.

Pipes follow the same edges the model uses. This is a constraint rather than a coincidence: per-edge attention returned by the network maps one-to-one onto drawn pipes only if the two lists agree. Connectors are routed as right angles that begin and end on box edges. Getting this right took a correction. The original routing produced a final segment that ran along a target box's own edge, so a pipe appeared to cut through the equipment. Routing the vertical run through the gap between the two boxes fixed it, and the geometry was verified numerically for every edge before being accepted.

Two overlays sit on top of the risk colour: diagonal hatching with a short tag where a permit is active, and dots indicating workers present. These are the situational-awareness layer, and they are what turn the map from a heat display into something that shows why a zone matters.

9.2 Views

Six views share the shell. Single run executes a scenario and shows the result. Time replay scrubs a run's risk timeline. Live CCTV runs the detectors against a clip or camera. Devices manages cameras and sensors. Evaluation presents the benchmark results. About describes the system.

Live mode is a control on the Single run view rather than a separate page, since it displays the same map with a different data source. It streams over `EventSource` and stops cleanly when the view changes.

9.3 Theming and interface state

Both light and dark themes are supported, driven entirely by CSS custom properties with a `[data-theme="light"]` override. Dark is the default and the choice persists.

Building the light theme surfaced a category of bug worth recording: several rules had hard-coded values that silently assumed a dark background. White overlay washes, a `#fff` label on what became a near-white active state, and pale text tuned for dark panels all became invisible or illegible. Making them theme variables rather than literals is what actually fixed it.

9.4 Chat memory

The operator chat carries a short rolling history so follow-up questions work in shorthand. The endpoint stays stateless; the thread lives client-side in `sessionStorage`, keyed per facility.

That storage choice is deliberate on two counts. The conversation survives a page reload, which is the common way an operator loses context mid-task, but it is gone when the tab closes, so plant questions are not left sitting on a shared control-room machine. Keying by facility prevents context from one plant leaking into a conversation about another.

Error bubbles are filtered out of the history sent upstream. They are interface feedback, not something the model said, and replaying them caused it to apologise for failures it had never produced.

10 Evaluation

All figures below come from a held-out test split of 120 runs, of which 30 are compound-positive. The graph model and the baseline are scored on the same split, at each run’s true hazard zone, with the split taken at run level so no episode straddles the boundary.

10.1 Detection quality

Model	Precision	Recall	F1
Compound-risk GNN	1.00	1.00	1.00
Single-sensor baseline	0.37	1.00	0.54

Table 5: Detection at the true hazard zone.

Recall is identical, and that is the point. The baseline reaches full recall by alarming almost continuously, which is what a precision of 0.37 describes. Roughly two out of three of its alarms are false. In a real control room that is how alarm fatigue begins, and it is the mechanism by which genuine warnings end up ignored.

10.2 False negatives at matched false-alarm budgets

Comparing at a fixed operating point is fairer than comparing headline accuracy, because it holds the nuisance cost constant and asks what each model misses.

Budget	GNN FNR	Baseline FNR
5% FPR	0%	90%
10% FPR	0%	80%
20% FPR	0%	60%

Table 6: Missed compound hazards at matched false-positive rates. Lower is better.

Allowed the same false-alarm budget a plant would tolerate, the baseline misses the large majority of real compound hazards. This is the result with the clearest human consequence, and it is the one we lead with.

10.3 Zone localisation

Detecting that something is wrong is only half of it. The model’s highest-scoring zone matches the zone the fault was injected into in 100% of compound test runs. The baseline manages 23.3%.

An alarm that names the wrong zone is worse than useless during an evacuation, so this number carries more operational weight than its absence from most published comparisons would suggest.

10.4 Lead time, stated honestly

Median confirmation latency is zero samples for both models. The graph network does *not* detect earlier than the baseline on this benchmark.

We state this plainly because an earlier version of the replay code appeared to show a lead-time advantage, and the advantage was an artefact. Permit and presence records were visible to the model at cutoffs before those records actually became active, so the model was reading the future. Once visibility was gated by the real `from_sample` values, the apparent advantage disappeared.

The defensible claim is therefore about precision and attribution, not anticipation: at the same moment the baseline fires with 37% precision and the wrong zone three times in four, the graph model fires with high precision and the right zone.

10.5 On the saturated metrics

A precision, recall and F1 of exactly 1.00 should invite scepticism rather than confidence, and we read it as a statement about the benchmark rather than the model. The test split contains 30 compound-positive runs generated from five scenario templates, and the injected faults are distinguishable enough that a model with the right inductive bias separates them completely. On messier data, with faults closer to the noise floor and more varied plant behaviour, these numbers would come down.

What the result supports is a comparative claim: given identical data, the graph formulation extracts a signal the threshold formulation cannot. It does not support a claim about absolute field performance, and Section 12 returns to what would be needed for that.

11 Deployment and Operations

11.1 One-command run

`docker compose up` brings up four services: PostgreSQL with pgvector, Ollama, the FastAPI backend, and an nginx-served frontend build. The backend entrypoint waits for Postgres, applies Alembic migrations, seeds idempotently, pre-warms the embedding model, and only then starts accepting traffic. Ordering matters: the API should not answer requests it cannot yet serve correctly.

A `Makefile` wraps the common operations. Two targets exist specifically because startup is slow and opaque: `make logs-backend` tails just the backend so the migration, seed and model-load phases are visible, and `make wait` blocks until the API genuinely answers rather than until Docker reports the container as started. Those are different moments, and conflating them causes confusion during a demonstration.

11.2 GPU and platform notes

A GPU overlay adds NVIDIA device reservations and raises the language model's layer offload. On Linux and Windows this requires the NVIDIA Container Toolkit, which is separate from the driver.

Apple Silicon is a genuine exception and the documentation says so. Docker cannot pass the Metal GPU into a Linux container, so the `compose GPU` overlay is meaningless there. Mac users get `setup-mac.sh` and `run-mac.sh`, which run the stack natively via Homebrew. That path also happens to be the only one where a built-in webcam is reachable, since neither Docker Desktop nor WSL2 exposes host cameras to containers without additional work.

11.3 Kubernetes and CI

A `kustomize` base defines the production deployment: a `StatefulSet` for Postgres, Ollama with a persistent model cache, the backend, two frontend replicas, and an ingress routing `/api`. A GPU overlay layers on device limits. The manifests validate clean under `kubeconform` in strict mode.

The GitHub Actions workflow runs checks, builds and pushes images to GHCR tagged with both `latest` and the commit SHA, then performs a cluster rollout gated on a secret being present, so a fork without credentials still runs the build. A Jenkinsfile mirrors the same stages for teams running in-house CI.

11.4 Two operational lessons

Vendored model weights. Fetching the embedding model at first use made a clean install unreliable in exactly the environments where reliability matters. It is now committed to the repository. This makes the image larger and is the right trade: a safety platform that requires internet access to complete its first request is not deployable on an isolated plant network.

Cache-friendly rebuilds. A full `-no-cache` rebuild re-executes `apt-get update` and `pip install`, both of which are network operations that can hang rather than fail on constrained connections. The default rebuild target now reuses those layers, since Docker already invalidates everything downstream of the source copy when source changes. A separate target remains for the rare case where a dependency genuinely changed.

12 Limitations and Further Work

12.1 What the evaluation does not establish

The benchmark is synthetic. Telemetry comes from a process simulation and the human layer is generated, so the joint distribution of process faults and work activity is one we constructed. The saturated metrics in Section 10 should be read as evidence that the graph formulation beats the threshold formulation on identical data, not as a prediction of field accuracy.

Validating properly needs historical data from a real plant: telemetry archives, the permit system, and incident records covering the same period. That is a data-access problem rather than a modelling one, and it is the single most valuable next step.

12.2 Known gaps

Demonstration CCTV. Two baked frames serve every scenario, drawn from a public hard-hat dataset. Adding variety needs the source dataset re-downloaded and more frames curated, with a mapping from scenario to frame. The live inference path is unaffected.

Vision and telemetry are only loosely coupled. Detections enter the reasoning layer as corroborating evidence attached to a violation. They do not enter the graph as nodes, so the model cannot learn that a particular visual condition raises risk in a particular zone. Making detections first-class graph nodes is the natural extension.

Fixed process topology. Zone-to-zone edges are hardcoded per facility template. For a plant whose configuration changes with operating mode, the topology should be data-driven, and possibly learned.

Single-window scoring. The model reads a 30-sample window and scores the present. It does not forecast. Predicting that a trajectory will become hazardous, rather than recognising that it now is, would be a genuine step toward the lead time this system does not currently claim.

12.3 Security posture

Authentication is a facility-scoped API key with per-device ingest tokens. That is proportionate to a pilot and insufficient for production. A real deployment needs per-user identity, roles distinguishing viewers from officers who can acknowledge alerts, and an access log alongside the existing audit trail. The multi-tenant schema supports this; the identity layer is simply not built.

Device tokens are also surfaced in the interface so the integration snippet can be copied directly. That is a deliberate trade for a pilot where dashboard access already implies operator trust, and it should be revisited before wider deployment.

12.4 Closing

The gap this project addresses is not a shortage of sensors. It is that the information needed to prevent an incident is routinely present and distributed across systems that never speak to each other. What we have built is one demonstration that fusing those sources is tractable, that the fused signal is measurably better than what per-channel alarms produce, and that the result can be made auditable enough for a safety context.

The honest summary is that the modelling approach is validated and the deployment story is real, while the evidence base is synthetic. Closing that last gap requires a plant willing to share its history.